

OTA (Over-The-Air) on ESP32

Remote Software Update: Principles, Use Cases and Implementation

Oussama Jhimi

December 14, 2025

Abstract

OTA (Over-The-Air) update allows deploying new firmware to a microcontroller without physical connection, via WiFi, Bluetooth or a remote server. On ESP32, OTA is natively supported by ESP-IDF and ArduinoESP32, making it an essential tool for modern IoT projects. This article presents motivations, OTA types, partition architecture, implementation examples, application domains and professional best practices.

1 Why use OTA

1.1 Main Advantages

- **Remote maintenance:** no need for physical access.
- **Mass deployment:** update hundreds of devices simultaneously.
- **Security:** quick fixes in case of vulnerabilities.
- **Cost and time saving:** no disassembly, no cables.
- **Improved user experience:** device remains operational.

1.2 OTA Security

- Firmware signature.
- Integrity verification (SHA256).
- HTTPS connection.
- Authentication (token, API key).

2 OTA Architecture on ESP32

ESP32 uses dedicated partitions in flash memory:

- **bootloader**
- **partition table**

- `ota_0`
- `ota_1`
- SPIFFS / LittleFS

Principle:

1. Current firmware runs on `ota_0`.
2. New firmware is downloaded into `ota_1`.
3. Bootloader switches to the new partition.
4. Automatic rollback in case of failure.

3 Types of OTA on ESP32

OTA Type	Description	Use Case
Local OTA (Web Server OTA)	ESP32 hosts a web page to upload firmware	Prototyping, local maintenance
OTA via WiFi + HTTP(S)	ESP32 downloads firmware from a server	Large-scale deployment
OTA via MQTT	Firmware download triggered by MQTT message	Industrial IoT networks
OTA via Bluetooth	Update via smartphone	Mobile devices, wearables

4 Implementation Examples (ArduinoESP32)

4.1 OTA via WiFi with ArduinoOTA

```

1 #include <WiFi.h>
2 #include <ArduinoOTA.h>
3
4 const char* ssid      = "Your_SSID";
5 const char* password = "Your_PWD";
6
7 void setup() {
8     Serial.begin(115200);
9
10    WiFi.begin(ssid, password);
11    while (WiFi.status() != WL_CONNECTED) {
12        delay(500);
13        Serial.print(".");
14    }
15    Serial.println("\nConnected to WiFi");

```

```

16
17  ArduinoOTA.setHostname("ESP32-OTA");
18  ArduinoOTA.setPassword("OTApasword");
19
20  ArduinoOTA.onStart([]() {
21      Serial.println("OTA Start");
22  });
23  ArduinoOTA.onEnd([]() {
24      Serial.println("\nOTA End");
25  });
26  ArduinoOTA.onProgress([](unsigned int progress, unsigned int
    total) {
27      Serial.printf("Progress: %u%%\r", (progress * 100) / total);
28  });
29  ArduinoOTA.onError([](ota_error_t error) {
30      Serial.printf("OTA Error: %u\n", error);
31  });
32
33  ArduinoOTA.begin();
34 }
35
36 void loop() {
37     ArduinoOTA.handle();
38 }

```

4.2 OTA via HTTP Server (update from URL)

```

1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <Update.h>
4
5  const char* ssid          = "Your_SSID";
6  const char* password      = "Your_PWD";
7  const char* firmware_url = "http://your-server.com/firmware.bin";
8
9  void setup() {
10     Serial.begin(115200);
11
12     WiFi.begin(ssid, password);
13     while (WiFi.status() != WL_CONNECTED) {
14         delay(500);

```

```

15     Serial.print(".");
16 }
17 Serial.println("\nWiFi OK");
18
19 HTTPClient http;
20 http.begin(firmware_url);
21 int httpCode = http.GET();
22
23 if (httpCode == HTTP_CODE_OK) {
24     int contentLength = http.getSize();
25     WiFiClient* stream = http.getStreamPtr();
26
27     if (Update.begin(contentLength)) {
28         size_t written = Update.writeStream(*stream);
29         if (written == contentLength) {
30             Serial.println("Firmware downloaded");
31         }
32         if (Update.end()) {
33             if (Update.isFinished()) {
34                 Serial.println("Update OK, reboot...");
35                 ESP.restart();
36             }
37         }
38     }
39 }
40 http.end();
41 }
42
43 void loop() {}

```

4.3 OTA via Integrated Web Server (manual upload)

```

1 #include <WiFi.h>
2 #include <WebServer.h>
3 #include <Update.h>
4
5 WebServer server(80);
6
7 void handleRoot() {
8     server.sendHeader("Connection", "close");
9     server.send(200, "text/html",

```

```

10     "<form method='POST' action='/update' enctype='multipart/form
    -data'>"
11     "<input type='file' name='update'>"
12     "<input type='submit' value='Update'>"
13     "</form>");
14 }
15
16 void handleUpdate() {
17     HTTPUpload& upload = server.upload();
18
19     if (upload.status == UPLOAD_FILE_START) {
20         Update.begin();
21     } else if (upload.status == UPLOAD_FILE_WRITE) {
22         Update.write(upload.buf, upload.currentSize);
23     } else if (upload.status == UPLOAD_FILE_END) {
24         Update.end(true);
25     }
26 }
27
28 void setup() {
29     WiFi.begin("SSID", "PWD");
30     while (WiFi.status() != WL_CONNECTED) delay(500);
31
32     server.on("/", HTTP_GET, handleRoot);
33     server.on("/update", HTTP_POST, []() {
34         server.send(200, "text/plain", Update.hasError() ? "Error" :
            "OK");
35     }, handleUpdate);
36
37     server.begin();
38 }
39
40 void loop() {
41     server.handleClient();
42 }

```

5 Application Domains

5.1 Industry & IoT

- Industrial sensors
- Connected PLCs
- Supervisory systems (SCADA)

5.2 Home Automation

- Smart light controllers
- Intelligent thermostats
- Alarm systems

5.3 Automotive

- Telematics modules
- Embedded dashboards

5.4 Prototyping & R&D

- Rapid update during testing phase
- Deployment on test benches

5.5 Remote Connected Devices

- Weather stations
- Drones
- Autonomous robots

6 Best Practices for OTA

6.1 Security

- Always use **HTTPS**.
- Verify firmware **signature** and **SHA256**.
- Employ strong **OTA passwords**.

6.2 Architecture

- Plan for **two OTA partitions**.
- Enable **automatic rollback**.
- Store the **current version** in NVS.

6.3 Monitoring

- Log all **updates**.
- Notify in case of **failure**.
- Maintain a **versioned firmware server**.

7 Conclusion

OTA is an indispensable feature for any serious ESP32 project. It enables maintenance, bug fixing and feature enhancement without physical intervention, while ensuring security, flexibility and scalability.